List of Features (0.25pts each — Total 10pts)

Group 52 — Super Mario Game. This project was scoped against `SPEC.md`'s expanded v2.0 specification (110 features across 20 categories) specifically because the team has two members instead of one — the brief in `FEATURE_PROPOSAL.md` justifies doubling the normal single-member scope. The list below is not the generic 40-item version; every line was checked against the actual source in `SuperMarioGame/include/` and `SuperMarioGame/src/` (not just `SPEC.md`'s intent) during a dedicated feature audit, and is reachable from a real playthrough — constructed through `EntityFactory`, wired into `PlayingState` / `Game`, and not merely exercised by a `verify_*.cpp` test harness. A small number of items are marked *(editor-authored)* — implemented, wired, and fully functional, but placed into the world through the in-game level editor, the debug console or the dev panel rather than by one of the seven shipped campaign/bonus levels; everything else is present in the campaign as shipped. That marker has **shrunk** since the previous edition: Koopa Paratroopa, Boo, Thwomp, Lakitu, Bullet Bill and Chain Chomp were all subsequently placed into the shipped levels and no longer carry it (verified by parsing `SuperMarioGame/assets/levels/*.json` — Paratroopa ×3 and Lakitu ×1 in `level_1.json`, Boo ×2 in `level_2.json`, Thwomp ×2, Bullet Bill ×3 and Chain Chomp ×1 in `level_3.json`). It has also been **added** where the same parse showed it was owed: no shipped level dispenses a Mini Mushroom or a Mega Mushroom (`QuestionBlock::Content` ids 5 and 6 appear in no level file), so those two are editor-authored and are now labelled as such.

Some SPEC.md-described behaviour did **not** make this list because the audit could not confirm it is reachable from the running game — for example, climbing/vines, the ground-pound hover pause and enemy-stun radius, the skid mechanic, input-locking on knockback, and the `kill_all` / `stats` debug commands. Listing those would have been the kind of optimistic claim this project's own `AGENTS.md` explicitly warns against. Three items have left this exclusion list since the original audit because they were subsequently built for real, not because the standard was relaxed: a true infinite **Endless Mode** (#96), the **dynamic lighting shader** with a day/night cycle (#98–#100 — the earlier edition of this document explicitly excluded lighting, and that exclusion is now wrong), and **noclip**, which exists as a Debug > Cheats switch rather than as the console command SPEC.md described (#105).

1. Core Engine & Architecture

1. **Fixed-Timestep Game Loop** — 1/60s accumulator with interpolated rendering, independent of frame rate; the same determinism the time-rewind and replay systems rely on.
2. **Game State Management (State Pattern)** — `GameStateManager` stacks 9 screens (Menu, Character Select, World Map, Playing, Pause, Options, Game Over, Victory, Level Editor); Pause/Victory overlay the level rather than replacing it. Nine is a counted figure, not a remembered one: `grep -rl ": public IGameState" SuperMarioGame/include/` returns exactly nine headers.
3. **Singleton Managers** — `Game`, `ResourceManager`, `SoundManager`, `InputManager`, `EventBus`, `AchievementManager` and 6 more (12 total — counted from the `getInstance()` declarations in `SuperMarioGame/include/`, not from

SPEC.md's older figure), Meyers singletons with defined construction order (`ResourceManager` deliberately uses a never-destroyed instance instead, so textures outlive every other static).

4. **Event System (Observer Pattern)** — `EventBus` with 29 event types (counted from the `EventType` enum) and six independent subscribers: `SoundManager` (20 subscriptions), `PlayingState` (14), `AchievementManager`, `StatisticsTracker`, `Camera` (7) and `Minimap`. The HUD is deliberately *not* a subscriber — it is refilled from a `HudData` struct every frame, which is why an eliminated player's badge can survive the player object itself (#118).
5. **Input Handling (Command Pattern)** — every action is an `ICommand` object (`JumpCommand`, `MoveLeftCommand`, `RunCommand`, `GroundPoundCommand`, `WallJumpCommand`, `CrouchCommand`, `FireCommand`…), rebindable and shared by Player 2 through a second binding table.
6. **Object Pooling** — `ObjectPool<T>` reuses fireballs, hammers and boss projectiles; particles are pooled separately (200 pre-allocated).
7. **Spatial Hashing** — 64px broad-phase collision grid; an $O(n^2)$ all-pairs test becomes $O(n)$ expected work against bucket neighbours.
8. **Config-Driven Entity Tuning** — `assets/config/entities.json` adjusts per-type speed and score at spawn time without a recompile.
9. **Debug Console** — backtick-toggled ImGui console with 11 real commands (`help`, `give`, `lives`, `tp`, `god`, `spawn`, `difficulty`, `level`, `progress`, `replay`, `clear`); the count is the number of `registerCommand` calls in `DebugConsole::registerBuiltins`.
10. **Data Serialization** — full JSON round-trip of player state, level progress, statistics, achievements and settings via `nlohmann::json`.

2. Physics & Collision

1. **AABB Collision Detection** — axis-aligned bounding boxes for every entity, checked through the spatial hash.
2. **Nine-Stage Kinematic Physics Pipeline** — conveyor push → interactive tiles → acceleration/friction → gravity → X-axis integrate/resolve → Y-axis integrate/resolve → map bounds → broad phase + narrow phase/resolution → intent-flag clear, in a fixed, load-bearing order.
3. **Collision Resolution** — dispatches by entity-pair type: stomp, side-damage, item collection, block bump, shell kick, boss contact, player-vs-player in multiplayer.
4. **Coyote Time & Jump Buffering** — 6-frame (100ms) forgiveness windows on both sides of a jump.
5. **Dual Bounding-Box Architecture** — a `physicsBox` for collision and a separate `combatBox` for hit detection, so a stomp and a solid-tile check never fight over the same rectangle.
6. **Surface-Dependent Physics** — ice tiles cut deceleration from ~1000 to 250 px/s²; conveyor tiles push at a constant 100px/s.

3. Advanced Movement Mechanics

1. **Wall Jump / Wall Slide** — capped 50px/s downward slide, launches off the wall on jump.
2. **Ground Pound** — 600px/s instant slam with screen shake and a stomp cue on landing.
3. **Crouch / Slide** — halves the player's hitbox, lets them pass under 1-tile gaps, and stands back up only when the ceiling is clear.
4. **Damage Knockback** — a fixed 150×100px impulse away from the hit, plus temporary invincibility with a visible sprite flash.
5. **Momentum & Acceleration Curves** — separate ground/air friction and deceleration rates, with ice further reducing grip.
6. **Combo System** — stomping enemies without touching ground increments a HUD-visible counter that resets after a 2.5s idle window or on taking damage.

4. Characters & Player States

1. **Four Playable Characters** — Mario (baseline), Luigi, Toad and Peach, each with distinct movement constants.
2. **Luigi** — ×1.2 jump force, ×0.85 walk/run speed, ×0.9 airborne gravity, plus a genuine double jump.
3. **Toad** — ×1.3 speed, ×0.8 jump force, and slide momentum that is not cut by ground friction.
4. **Peach** — ×0.9 speed and a float/hover that lasts up to 1.5s.
5. **Unlockable Characters** — Toad and Peach open on real `AchievementManager` gates (finish the campaign; finish it without dying) and become selectable at Character Select.
6. **Player Forms (State + Decorator Patterns)** — five concrete `IPlayerState` forms (Small, Super, Fire, Cape, Mini) with a full power-up/power-down transition chain.
7. **Star Invincibility (Decorator)** — `StarDecorator` wraps whichever form is active for 10 seconds without altering it, so a Star picked up as Fire Mario still returns to Fire Mario after.
8. **Mega Mushroom (Decorator)** *(editor-authored)* — `MegaDecorator` scales the player up and grants 8 seconds of damage immunity. Dispensed by a Question Block whose content id is 6, by the dev panel's spawner, or by `give mega`; no shipped level places one.
9. **Two-Player Same-Keyboard Input** — Player 1 on WASD (+F fire, Q ground-pound, LShift run, Space jump), Player 2 on the arrow cluster (+Period or RControl fire, Slash ground-pound, N run, Up or RShift jump), both routed through the same Command-pattern pipeline. Player 2's fire key was moved off `M` because `M` toggles the minimap (#68); the two bindings used to collide.

5. Enemies & AI

1. **Entity Factory (Factory Pattern)** — `EntityFactory::create` is the single construction point for 25+ entity types from a level-file string, with no `#include` of the concrete class required at the call site.
2. **Eight Movement Strategies (Strategy Pattern)** — Patrol, Chase, Fly, TimerEmergence, Linear, HammerThrow, TetheredChase and ProximityTrigger, swappable per enemy at construction.
3. **Goomba & Koopa Troopa** — classic ledge-aware patrol, with Koopa shells that can be kicked, carried and used as a weapon.
4. **Koopa Paratroopa** — vertical-bounce and sinusoidal-patrol flight variants; three are placed over the mid-level platforms of World 1-1, and the level editor's palette can place more.
5. **Piranha Plant** — timer-based emergence from a pipe, implemented via `TimerEmergenceStrategy`.
6. **Hammer Bro** — tracks the player's side and throws a rotating hammer projectile roughly every 1.5s.
7. **Boo** — freezes when faced, pursues at 80px/s the moment the player looks away; immune to stomping and fireballs. Two haunt the Ice Cavern of World 1-2.
8. **Thwomp** — teeth-gritting wind-up, a 600px/s slam, a floor rest, and a slow climb back to its start height. Two guard corridors in Bowser's Castle.
9. **Lakitu & Spiny** — Lakitu flies overhead above World 1-1 and spawns Spiny enemies via the same Factory the rest of the game uses (through an `EntitySpawnRequested` event, so no enemy ever grows the entity list mid-iteration); Spiny patrols the ground, and four are also placed directly in Worlds 1-2 and 1-3.
10. **Bullet Bill & Chain Chomp** — a straight-line projectile enemy and a tethered lunge-and-recover enemy; Bowser's Castle places three Bullet Bills and one Chain Chomp.

6. Bosses

1. **Boom Boom (World 1-2 mid-boss)** — a three-stomp fight with an escalating charge/spin/recover pattern per phase, health-scaled by difficulty.
2. **Bowser (World 1-3 final boss)** — two-phase fight: Phase 1 walks and breathes fire; Phase 2 adds a leap and a faster (1.2s vs 2.2s) fire cadence. Fireballs do not damage him but do accumulate: the fourth landed fireball **staggers** him for 3

seconds, and a staggered Bowser can be stomped safely and without invulnerability frames — the HUD counts the fireballs still owed.

3. **Boss Template (Template Method Pattern)** — `Boss::update()` is sealed and calls a pure-virtual `updateBehaviour()`, so every boss gets its invulnerability frames, phase transitions and defeat sequence in the right order regardless of its attack pattern.
4. **Boss Arena Lock** — "no escape until defeated": the camera locks to the arena and the player's position is clamped inside it until the boss's health reaches zero, gated independently of level geometry so a misplaced arena cannot skip the fight.
5. **Boss HUD** — a dedicated health bar and boss name appear only while a boss fight is active.

7. Items & Power-ups

1. **Super Mushroom, Fire Flower, Cape Feather, Mini Mushroom, Mega Mushroom, Star, 1-Up Mushroom** — seven power-ups, each driving the same `Player::powerUp()` state-transition path. Five of the seven are dispensed by shipped levels; Mini and Mega are *(editor-authored)* (see the note above #30).
2. **Coins & the 100-Coin 1-Up** — collecting 100 coins grants an extra life.
3. **POW Block** — flips every grounded, on-screen enemy when hit.
4. **P-Switch** — a 15-second window that swaps bricks and coins level-wide.
5. **Trampoline** — configurable spring bounce, placed throughout the campaign and all three sub-vaults.
6. **Star Coins** — exactly 3 per main/bonus level (1 per sub-vault), tracked per level and persisted across sessions.

8. Blocks & World Objects

1. **Question Blocks & Brick Blocks** — bump-to-reveal item logic and break-from-below physics tied to the player's current form.
2. **Warp Pipes** — bidirectional transitions between a main level and its underground/sky/lava sub-vault, preserving lives, coins, score and power-up state.
3. **Moving Platforms** — patrol back and forth and carry the player riding on top, with their own sprite animation advancing as they travel. `MovingPlatform::update()` fully overrides `Block::update()` to own its kinematics and for a long time never called the base version, so every placed platform moved with a frozen first animation frame; it now delegates, and the fix is guarded by a mutation-tested regression case that advances a platform for half a second and asserts the animator moved.
4. **Falling Platforms** — a four-state cycle (Idle → Shaking 1s → Falling → Respawning 5s).
5. **Ice & Conveyor Tiles** — level-authored surfaces that feed directly into the physics pipeline's surface-dependent friction and push.
6. **Flagpole Completion** — score awarded on a five-tier scale (100 to 5000) based on how high the player is caught, gated so it cannot fire while a level's boss is still alive.
7. **The End-of-Level Castle** — a decorative structure whose flag climbs the gatehouse in step with the flagpole's own descent; the player visibly walks up to its door before the summary screen appears.

9. Levels & World

1. **A Three-World Campaign** — World 1-1 (Grassland), World 1-2 (Ice Cavern, Boom Boom), World 1-3 (Bowser's Castle Fortress), plus a selectable Bonus Stage, all authored in JSON and loaded through `LevelLoader`.
2. **Three Sub-Vaults** — an Underground vault (1-1), a Sky Canopy vault (1-2) and a Secret Lava vault (1-3), each a self-contained side room reached and left through a warp pipe.
3. **World Map** — sequential level unlocking, a per-node star-coin pip display and a completion tick, navigated with Left/Right/Enter.
4. **Procedural Level Generator (`MapGenerator`)** — multi-tier elevation, biome-specific ceilings, lava pits, reachability-guarded platform gaps and a threat-pacing curve; drives both the Daily Challenge and a "Generate & Play/Edit" menu

page.

5. **Level Editor (Mario Maker)** — a screen of its own, `EditorState`, entered from the main menu's MAP EDITOR row rather than as an overlay toggled with F1 (which is what an earlier edition of this document described). Six tools on single-key shortcuts — Paint (`B`), Erase (`E`), Rect Fill (`R`), Eyedropper (`I`), Select (`V`) and Spawn Point (`P`) — a tile palette and an entity palette built from the registry (#104), an inspector for the selected entity's properties, `Q` / `T` to step the palette, `Delete` to remove the selection, mouse panning, and `Ctrl+N` / `Ctrl+O` / `Ctrl+S` / `Ctrl+Shift+S` for new, open, save and save-as against `assets/levels/custom/`.

10. Game Flow & UI

1. **Main Menu** — ten rows: Start Game, Load Game, Multiplayer, Daily Challenge, Map Editor, Custom Levels, Procedural Level, Records, Options and Quit, with an animated parallax background and a walking-Mario idle sprite. Five of the rows carry a live status value beside the label rather than being static text — the New Game+ cycle on Start Game, "4 MODES" on Multiplayer, today's generated challenge name, the number of custom levels found on disk, and unlocked/total achievements on Records — so the menu reports state without the player having to open the page first.
2. **Pause Menu** — Resume, Save Game, Options, Restart Level and Quit to Menu, overlaying the still-visible level.
3. **HUD** — live score, coins, world/level, timer, lives, combo counter, and Star Coin indicators.
4. **Minimap** — an overview of the whole level and every live entity, colour-coded and colourblind-palette aware, toggled by **either** `M` **or** `Tab`. `M` is the primary key named in SPEC.md; `Tab` is kept as an alias. The minimap subscribes to the `MinimapToggled` event itself rather than being reached through a getter, so the key handler knows nothing about it.
5. **Statistics Screen** — cross-session totals for coins, enemies defeated, deaths, combo hits and play time, persisted to a profile file.
6. **Screen Transitions & Camera Feel** — fade in/out between levels, a velocity-driven camera lookahead, and event-triggered screen shake (block break, ground pound, POW block).
7. **Death & Retry** — a global death tally and an instant "Retry Level" that skips menu navigation entirely.

11. Audio

1. **Full SFX Coverage** — jump, coin, stomp, power-up, damage, fireball, player death, flagpole, pipe warp, ground pound, P-Switch and achievement cues, each fired from the real event that causes it.
2. **Adaptive Background Music** — distinct tracks for the menu, world map, overworld/underground/underwater/bonus themes, Star Power, boss fights and Game Over, swapped through `SoundManager`.
3. **Volume Controls** — independent music/SFX sliders in Options, persisted to `config.json`.

12. Save/Load & Persistence

1. **Three Save Slots** — full player/level/progress/statistics/settings state, written as JSON; loadable from the main menu's LOAD GAME picker, which previews character, level, score, star coins and play time per slot (or labels a slot Empty), confirming through the same `PlayingState::loadFromSlot` path the dev panel's Save/Load Slots tool already used.
2. **Auto-Save at Checkpoints** — a `CheckpointActivated` event silently persists progress; currently only the debug checkpoint key publishes it (the warp-pipe trigger was deliberately removed as a defect fix, so pipes are side-effect free).
3. **Manual Save** — "Save Game" from the pause menu, writing to the slot the run is attached to; the LOAD GAME picker (#75) and the slot delete (#126) are the other two ends of the same three-slot store, and all three go through `Serializer`'s own path resolution rather than assembling `saves/slot_n.json` themselves.
4. **High Score Table** — recorded per run (score, coins, star coins, character, level) and viewable from Options.

13. Two-Player & AI Opponents

1. **Two-Player Versus** — shared-keyboard local multiplayer with configurable stomp-vs-bounce collision between the two players. What happens once one of them runs out of lives is #118.
2. **Co-op Mode** — the same shared-keyboard setup, but vertical player contact becomes a cooperative boost instead of a bounce.
3. **AI Opponent (Versus CPU)** — a CPU-controlled second player with distinct behaviour profiles (e.g. a Speedrunner that climbs away vs. a Hunter that reverses to chase).
4. **Shadow Mario (Rival AI)** — replays the human player's own input history after a configurable 0.5–10s delay, complete with its own jump-replay logic — a full Memento/Command-pattern rival, not a scripted opponent.

14. Controls

1. **Full Rebinding** — every action can be reassigned from Options, with conflicting bindings swapped rather than silently orphaned, and persisted to `config.json`.
2. **Command-Pattern Input Layer** — the same `ICommand` objects back keyboard input, the debug console's dispatch, and replay serialization.

15. Visual Effects

1. **Particle System** — pooled particle bursts for brick fragments, coin sparkle, death poofs, stomps and combos, each fired from the `EventBus` event that causes it rather than from the entity that caused it. The three remaining, non-event-driven emitters are #125.
2. **Entity Death Animations** — a Goomba's timed squish-then-vanish, a fireball-killed enemy's upside-down flip-and-fall, and the player's own jump-then-fall death arc (a freeze, then a hop, then a fall that stops colliding with terrain and ignores input until it clears the view). A star-powered kill gets a fourth animation instead: #124.
3. **Invincibility Flash** — the player's sprite alternates alpha for the full 2-second post-hit invincibility window.
4. **Event-Triggered Screen Shake** — light/medium/heavy presets tied to specific gameplay events (block break, ground pound, POW block).

16. Advanced Systems

1. **Time Rewind (Memento Pattern)** — `TimeRewindManager` holds a 300-frame circular buffer of `GameSnapshot`s and restores the game to any of the last five seconds on demand.
2. **Replay Recording** — every level records a full playable trace automatically, savable/loadable through the debug console.
3. **Difficulty Modes** — Easy/Normal/Hard scale starting lives, enemy speed, level timer and boss health together through a single `IDifficultyStrategy`.
4. **New Game+** — clearing the campaign starts a new cycle with a real enemy-speed multiplier, while keeping unlocks and the completion counter.
5. **Daily Challenge** — a date-seeded procedural level, deterministic for every player on the same day.
6. **Achievement System** — ten tracked milestones (first stomp, level completions, secret finds, coin totals and more) driving toast notifications and character unlocks.
7. **Colorblind Mode** — an Okabe-Ito-derived palette swap applied to the minimap and debug overlays.
8. **Endless Mode** — a true infinite runner, not just a wide procedural level: the tilemap starts as one generated chunk and grows a fresh chunk of rising difficulty each time the player nears the current edge, forever, with no flagpole and distance travelled as the score. The level countdown is suspended for the whole mode — an endless run used to end, every time, when the shared 300-second clock expired on a level that has no end — and the HUD says so instead of counting down. Reachable from Main Menu → Procedural Level → Play Endless. Its boss rhythm is #127.

9. **Level Solvability Oracle** — every procedurally generated level or Endless Mode chunk is checked by an independent reachability pass (bounded by the game's own jump/run physics constants), with automatic reseeded retries if a generated layout fails the check; if all bounded attempts fail, the last layout is kept and the failure logged rather than blocking play.

17. Dynamic Lighting & Day/Night

1. **Dynamic Lighting Shader** — a GLSL fragment shader (`assets/shaders/radial_light.frag`) composited over the finished world pass every frame by `PlayingState::renderLightPass` . Up to eight simultaneous radial lights, each with its own radius, intensity and shadow tint: the player carries a ~9-tile lamp that breathes, every live fireball is its own moving ember, and the free camera (#108) carries one so a detached shot is not framed in the dark. Darkness is per-theme, so an Ice Cavern or a lava vault is dim at any hour while an overworld is not. If the machine reports no shader support the renderer says so once on stderr and the game draws normally — the feature fails off, never half on.
2. **Day/Night Cycle** — the overworld darkness is driven by a 100-second cycle whose phase is computed from level-elapsed time, so an outdoor level visibly passes from day to night and back while it is being played. Indoor themes are deliberately excluded from the clock rather than blended with it: a cave is as dark at noon as at midnight, and keeping the two independent is what makes each separately testable.
3. **Lighting Tunables Panel** — every number behind #98 and #99 is a live ImGui slider under Debug > Lighting: player lamp radius, its breathe amplitude, its shadow tint as an RGB picker, fireball radius and intensity, free-camera radius, a freeze switch for the day/night clock and a phase scrub for it, plus a one-click reset to the shipped defaults. The panel reports up front whether a GPU is actually behind any of it and shows the live phase, night factor and resulting darkness, so a value that looks wrong can be told apart from a shader that never loaded. Edits are queued and applied between frames rather than written into a running simulation.

18. The Level Editor & Custom Levels

1. **Editor Undo/Redo Command Stack (Command Pattern, with undo)** — every edit is an `IEditorCommand` with `execute` , `undo` and `describe` , and each of the seven concretes (place tile, erase tile, fill rectangle, place entity, erase entity, move entity, set entity property) stores its own inverse at construction. Paired undo/redo stacks and a History panel that lists what each entry did; `Ctrl+Z` , `Ctrl+Shift+Z` and `Ctrl+Y` at the keyboard, deliberately suppressed while a text field owns the keyboard so `Ctrl+Z` in the level-name box undoes typing and not a brush stroke.
2. **Custom Level Library** — `LevelCatalog` scans `assets/levels/custom/` and the main menu's CUSTOM LEVELS page lists what it finds, showing each level's authored name with its file stem beside it so two levels that named themselves the same are still distinguishable. A custom level is launched **by path**, deliberately not by a campaign index, so authoring a level cannot renumber the campaign or disturb per-level progress arrays. The row on the main menu carries the count, and an empty library says where levels come from ("MAP EDITOR > SAVE AS") rather than showing nothing.
3. **Editor Playtest Round-Trip (F5)** — `F5` in the editor writes the document under edit to a scratch file and pushes a real `PlayingState` over the editor, so the level is tested by the actual game rather than by a preview. Every way out returns to the editor with the document intact: quitting, finishing the level, **and dying** — a playtest that ends in Game Over used to route through the ordinary `GameOverState` and drop the player back at the main menu with the editor stranded underneath it. A playtest is also excluded from the high-score table and from achievement unlocks, because a scratch level is not a run.
4. **Entity Registry (Registry / Type Object)** — one table (`EntityCatalogue`) declares all **42** `EntityType` values with the name used in level JSON, the display label, the palette category and the function that constructs the type. The level parser, the editor palette and `EntityFactory::create` all read this one table; the factory's 42-case switch is gone. This is what makes the editor's palette complete: the same list was previously hand-written in four places and had drifted to 40/40/16/16 entries, so 24 palette buttons silently placed nothing. A regression case walks `0..EntityType::Count` and fails the build for any enumerator with no row, so a type added to the enum cannot become a dead button.

19. Capture & Debug Tooling

1. **Debug Cheats** — eight switches (Immortal, Invincible, Infinite Lives, Freeze Timer, Hide HUD, Noclip, Free Camera, Infinite Fireballs) plus a simulation time-scale dial from 0.1× to 2.0×, exposed both as checkboxes with tooltips under Debug > Cheats and as `F2` – `F11` hotkeys for use mid-capture. Everything is gated behind Options > Debug Mode: disarming answers every query "off" without forgetting the layout of switches. The time scale is applied to the simulation clock only, so input and the panel itself stay responsive at 0.1×.
2. **Immortality That Rescues Instead of Suppressing** — `IMMORTAL` does not merely swallow the kill; it puts the player back on solid ground and states why. The check sits at the one door every lethal event funnels through, ahead of the `PlayerDied` publish, so a rescued run is not also counted as a death by the statistics tracker — and a player who falls into the void is returned to the level instead of falling forever with no way back, which is what suppressing the kill on its own produces.
3. **Tainted-Run Rule** — the moment any cheat is switched on, the run is marked tainted, and it stays marked even if the cheat is switched back off, because the run was already affected by then. Five independent readers honour it: the achievement manager unlocks nothing, the statistics tracker records nothing, and the Game Over and Victory screens write no high score and no campaign completion. Cheats are cleared when a level run begins and again when it ends, so slow motion cannot follow the player out into the menus and a demo take cannot leave the next run dishonest.
4. **Free Camera** — detaches the view from the player entirely and pans it with WASD or the arrow keys at 650 px/s, pre-empting every follow path including the versus camera so the frame is not dragged back on the same tick. Level bounds are lifted while detached, so a shot can sit past the edge of the level; the invariant is re-asserted the moment the cheat goes off. Deliberately the same keys and the same speed as the level editor's panning rather than a second set to remember. The lone-player camera tether (#117) is suspended while it is on, so the player is not shoved back into a frame that was moved away on purpose.
5. **Console Tab-Completion** — `Tab` in the debug console completes the command name. One match completes straight through to its arguments; several complete only as far as every candidate agrees (the longest common prefix) and then list the candidates rather than guessing between them. Completion stops at the first space, because past that the text is arguments with no fixed vocabulary.

20. Warp Pipe Geometry

1. **Pipe Entry Modes** — a warp pipe declares whether it is entered from the top (stand on the rim, press Down), from its west face or from its east face (walk into the mouth), and the mode is both serialized in level JSON and drawn: a side-entry pipe renders as an L-bend whose shaft runs off the top of the frame with a mouth at floor level, which is the only art in the game that says "enter here, it goes *up*". The band a player must be standing in to enter is computed from the renderer's own mouth fraction rather than restated as a number, so the mouth that can be entered is by construction the mouth that is drawn, and the trigger measures feet so every player form enters the same mouth. Generated levels honour this too — the procedural generator's vault **exit** pipe is now side-entry, matching the hand-authored pairing, where it used to be a top-entry stub that read as a dead end.
2. **Pipe Entry Animation** — entering a pipe is not an instant cut. The player is walked or slid into the mouth over a timed tween while the screen fades, and only then does the sub-level load; the collision pass is suppressed for the duration so nothing pushes the player back out of the pipe they are entering.

21. Boss Fight Structure

1. **Bowser's Bridge and Lava Death** — reaching the axe at the end of the bridge publishes `BridgeChopped`; the bridge span drops into the lava and takes Bowser with it. The boss stops being physics-driven the moment he reaches the lava and burns down there in a scripted sequence, rather than being carried straight through the two lava tiles by gravity and left falling forever. The "boss wandered off the level" safety check explicitly refuses to fire while a lava death is in progress, so the two never race.
2. **Bridge Axes Scaled by Difficulty** — the number of axes the fight demands is the difficulty knob: one on Easy, two on Normal, three on Hard. **More** axes is harder, not easier — every axe has to be reached before the bridge goes, and only the last one chops, so a Hard fight means three crossings of the bridge under Bowser's fire instead of one. The quota is

sized against the axes inside the live fight's arena only, so an uncollected axe from a boss two Endless chunks back cannot satisfy the current one, and an axe dropped into a running level by the editor (which the setup pass never saw) chops immediately rather than deadlocking.

3. **Procedural Boss Arenas** — the level generator can build a full boss encounter, arena plus landing apron, and place it at the right-hand end of the playable run so the flagpole is the reward for getting past the boss rather than a detour around him. Every generated Hard-difficulty level gets one automatically; a half-built arena is refused outright, because an arena that does not fit is worse than the flat exit it would replace.

22. Enemy Behaviour Detail

1. **Lakitu's Mercy Fire Flower** — a Lakitu that has already filled its Spiny quota drops a Fire Flower on a timer instead. The drop is resolved *before* the Spiny cap is consulted, deliberately: a player pinned under a cloud by three live, unstompable Spinies is exactly the player the drop exists for. Only a drop that actually happened restarts the clock, so the timer cannot bank and then fire a volley.
2. **Spiny Eggs** — a Spiny comes into being as an egg. Lakitu tosses it sideways, gravity carries it down, and it hatches into a walking Spiny on ground contact, with a distinct egg sprite and animation until then. The egg state existed in the class for a long time as unreachable dead code — a dropped Spiny appeared already hatched and walking in mid-air — and hatching is now the default way the type is constructed, so it applies to every construction path rather than needing each caller to remember a flag.

23. Camera & Simulation Budget

1. **Lone-Player Camera Tether** — a single player is held inside the camera's own view on the X axis, so outrunning the camera's lag and lookahead cannot carry them off the side of what is on screen. Only X: the bottom edge is deliberately left unclamped, because the void kill plane sits well below the level and a falling player must keep dropping until that check catches them rather than being arrested at the bottom of the frame. Suspended under Free Camera (#108) and under Noclip, and written against whichever participant is actually alive, so the survivor of a two-player match is tethered too.
2. **Two-Player Elimination and Survivor Camera** — when one player of a two-player match runs out of lives they are eliminated rather than ending the match, and the camera follows the survivor for the rest of it. Their identity, final coins and final score are captured on the last frame anything can be asked of them, so the HUD keeps showing an **eliminated badge** naming that player afterwards — the survivor is told the match has gone one-sided instead of the panel simply disappearing. Elimination does not carry across a level load, so a player is never locked out of a level they were never eliminated in.
3. **Off-Camera Entity Update Gate** — entities outside a thinking region (the visible view plus half a view of margin in each direction — about 20 tiles horizontally at the shipped resolution) have their `update()` skipped, which is what makes a long Endless run affordable when every chunk's enemies are still in the entity list. The gate lives in the state that owns the camera, not inside each entity, because "am I worth simulating" is a question about the camera. What is exempt is a **whitelist**, so a category added later is safe by default: players (Shadow Mario trails the human by far more than any margin), projectiles (their own update is what expires them and returns them to the pool) and blocks (moving platforms sweep a path the player returns to) are never frozen, and neither is an enemy that does not collide with tiles, because nothing in the world would stop it coasting on a stale velocity. The panel reports the live census — thought, frozen, and exempt — so the saving is measurable rather than asserted.

24. Presentation & Audio Detail

1. **Attract Mode** — left alone on the main menu for 30 seconds (or on `F5` immediately), the game demonstrates itself: it launches a real `PlayingState` and drives it from a bundled recorded demo through the replay system, so the attract loop is the actual game being played rather than a video or a scripted set piece. Any key returns to the menu. Gated to the main page only, so idling on the Load or Multiplayer page does not interrupt what the player was choosing, and if the demo asset is missing or empty the game says so and returns to the menu instead of leaving an attract "demo" that plays

nothing. The idle threshold is overridable by environment variable, which is how it is exercised in scripted runs without a 30-second wait.

2. **Surface-Dependent Footsteps** — a walking player produces a footstep cue on a cadence, chosen from the tile actually under their feet: grass on the overworld ground, a metallic step on ice and metal, and a generic floor step on everything else. Both players have their own cadence timer, so two people walking do not sync into one sound, and the footstep bus is mixed lower than the gameplay cues so it stays ambient rather than competing with them. Airborne, crouching and wall-sliding players are skipped.
3. **Per-Row Menu Cues** — moving the highlight in a menu plays a short blip on every row change, not only when a move is blocked. A blocked action — a locked card, the end of a list, confirming an empty save slot — reuses the same short cue, and a **destructive** action gets a different one: deleting a save plays the block-break sound rather than a positive chime, and a delete that removed nothing sounds like the no-op it was.
4. **Escalating Combo Audio** — the combo hit cue rises in pitch with the combo, 12% per step, capped so a long chain does not run off the top of the scale. SPEC.md named four dedicated combo sound files that the asset set does not contain; pitching the existing cue is the honest substitute and is implemented as an `EventBus` subscription on `ComboHit` inside `SoundManager`, so nothing in the gameplay code knows the audio escalates.

25. Visual Effects, Continued

1. **Star-Power Kill Effect** — an enemy killed while the player has Star power gets a continuous 720° spin launch and a sparkle burst instead of the ordinary flip-and-fall, distinguishing a star kill from a fireball kill on screen. The kill method is not threaded through the defeat event (which carries only a score value); the effect is chosen by asking which player currently has Star power, which is sound because the event is published synchronously from inside the same collision resolution that granted the kill.
2. **Contextual Particle Emitters** — three particle types that are not driven by a gameplay event but by where the player is and what they are doing: dust off the wall during a wall slide, and ambient bubbles or lava embers emitted according to the hazard tile the player is standing in. Together with #85's five event-driven bursts, all eight declared particle types are reached by the running game.

26. Save Management & Endless Refinements

1. **Save-Slot Delete with Confirmation** — a slot can be deleted from the LOAD GAME picker, through a dedicated confirmation page that defaults to "keep". The path is resolved by `Serializer`, not rebuilt at the call site — the one time this project hand-rolled a save path it deleted the developer's real files. The active-slot pointer is deliberately left aimed at the slot just emptied, because that is where the checkpoint autosave writes and re-pointing it at a surviving save would let the next autosave quietly overwrite a file the player never touched. The picker re-reads from disk afterwards rather than clearing its cached preview, so a delete that failed shows as a delete that failed.
2. **Endless Boss Milestones** — Endless Mode places a boss arena on a fixed chunk interval and nowhere else. Because the difficulty ramp turns Hard from the seventh chunk onwards, and the generator drops a Bowser into every Hard map, a long run used to splice another Bowser into the level every hundred tiles, each one breathing fire on sight, against a fight system whose invariant is one boss per level. A boss is now a milestone: it is the only thing in the mode that is not more of the same, and meeting one is an event rather than the weather.

Scope note for graders

Every numbered item above was checked against the running source, not assumed from `SPEC.md`'s wording — several `SPEC.md` claims were found to be overstated during the audit (an exact power-of-two combo curve, a three-mode colourblind system, a per-level death counter, mirrored New Game+ levels) and were **deliberately left off this list** rather than claimed. "Endless Mode" (§19.5) was one of those gaps at audit time — SPEC.md described it but the codebase only had single-shot procedural generation — and has since

been implemented for real (#96) rather than left as a known gap; #97 is a byproduct of that work, a genuine improvement ported from an abandoned side branch's better idea (see `logs/agent_history.log`) without its GAN/RL framework. What remains is **127** features that a grader can point at in the code and, in nearly every case, reach by simply playing the shipped campaign.

How the 30 items added since the previous 97-item edition were admitted. Each candidate had to have a call path traced from `main()` — through `Game::run` and `GameStateManager` into the state that runs it — before it was written down. A class that compiles, and even one that passes a `verify_*` harness, was not enough; this project's own `AGENTS.md` has a dedicated rule about that, written after an audit found six subsystems marked complete that the running game never constructed. Candidates that failed the test were **dropped rather than softened**:

- `DeathEffectType::PlayerDeathHop`, the floating player-death overlay, is constructed only by `tests/verify_graphics_visual.cpp`. Nothing in `src/` ever asks for it. The player's death arc that #86 *does* claim is a different mechanism (`Player::m_dying` plus the death-fall phase in `PlayingState`), which is why #86 stands and the overlay is not listed.
- `UiRenderer::wrapText` had no production caller and was **deleted** rather than documented.
- The `kill_all` and `stats` console commands SPEC.md describes do not exist; the console registers eleven commands and those are not among them.
- The build-time single-asset-tree guard and the entity-registry parity test are regression guards, not gameplay, and are described in the report's process section instead of being counted as features here.

Two numbers in this document changed because they were recounted rather than remembered (`AGENTS.md` requires computed counts): the singleton total went from 13 to **12** and the event-type total from 28 to **29**, both counted from the source at the commit this edition was written against.